

JomCode Coding Bootcamp

Functions, parameters and scope

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Write small reusable functions with clear inputs and return values.
- 02 Understand positional/keyword arguments, scope and defaults.
- 03 Separate computation from terminal input and display.

Plain meaning → technical view → use

PLAIN MEANING

Give a repeated calculation or rule a clear name.

TECHNICAL VIEW

Functions accept parameters, use local scope and return values.

WHEN YOU USE IT

Reuse a seat-availability calculation in more than one route.

TIP

Print is for a person; return gives a value to the caller.

Worked example 1: Return versus print



chapter_09_functions_parameters_and_scope.py

```
1 def remaining(capacity, registered):  
2     return capacity - registered  
3  
4 value = remaining(20, 8)  
5 print(value)  
6 print(remaining(registered=3, capacity=5))
```

PREDICT Can a printed message replace a return value?

Example 1: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

12		
2		

WHY THIS HAPPENS

The caller needs a value to make the next decision. Printing is a separate side effect.

Exercise 1: Remaining-seat function

BUILD

Write `remaining_seats(capacity, registered)` returning the difference. Do not print inside it.

CHECK

20,8 => 12. 5,5 => 0. Caller can reuse the result.

IF STUCK

Keep presentation outside the function.

Worked example 2: Default argument



chapter_09_functions_parameters_and_scope.py

```
1 def greeting(name, prefix="Hello"):
2     return f"{prefix}, {name}"
3
4 print(greeting("Ada"))
5 print(greeting("Ada", prefix="Welcome"))
```

PREDICT Which call uses the default?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

Hello, Ada		
Welcome, Ada		

WHY THIS HAPPENS

The first omits prefix. Both functions return a value without reading hidden state.

Exercise 2: Normalizer

BUILD

Write `normalize_email(text)` with `trim` and `lowercase`, preserving the input binding.

CHECK

" ADA@EXAMPLE.COM " => `ada@example.com`. Empty => `empty`.

IF STUCK

Return the transformed value.

Worked example 3: Safe mutable default



chapter_09_functions_parameters_and_scope.py

```
1 def add_title(title, titles=None):
2     if titles is None:
3         titles = []
4     return [*titles, title]
5
6 print(add_title("Python"))
7 print(add_title("SQL"))
8 print(add_title("React", ["Python"]))
```

PREDICT Why use None and a new list?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
['Python']  
['SQL']  
['Python', 'React']
```

WHY THIS HAPPENS

Each omitted-list call remains independent. The function also preserves an explicitly supplied list.

Exercise 3: Independent collections

BUILD

Write `with_registration(ids, participant_id)` returning a new list. Duplicate ID leaves values unchanged.

CHECK

`[p1],p2 => [p1,p2]. [p1],p1 => [p1].` Source remains `[p1]`.

IF STUCK

Return a new outer list in both branches.

Checkpoint

PROVE IT

Reusable domain functions have explicit inputs, useful returns and no hidden state.